

"The best way to learn to code is to write code, every single day."

FLUTTER & DART

DEVELOPER INTERVIEW & PRACTICE GUIDE

60 DAYS, DAY-BY-DAY: LEARN,
BUILD A PROJECT, GET TIPS

RAJENDRA PANDEY

How to Use This Guide

This guide takes you from complete beginner to a job-ready Flutter developer in 60 focused days. Every single day follows the same rhythm: **learn the concept**, then **build a small project** that uses it.

Each day is laid out the same way so you always know what to do. You get the **day's topic**, a **What you'll learn** section that explains every point in plain language, and a **Daily Project** with a description of what to build, the exact steps to follow, and practical tips and suggestions.

The journey covers everything: the Dart language, object-oriented and asynchronous programming, the full Flutter widget system, navigation, forms, state management (setState, Provider, Riverpod, Bloc), networking and JSON, local storage, Firebase, animations, testing, performance and deployment - finishing with a two-week capstone and dedicated interview prep.

Daily routine (about 2-4 hours): read the topic, type the project yourself (never copy-paste), get it running, commit it to GitHub, and jot down anything unclear. Tooling here targets the current stable line (Flutter 3.x / Dart 3.x).

At the end you'll find **45 interview questions with answers** (Part 2) and **7 quizzes with answer keys** (Part 3). Use the quizzes as weekly checkpoints - if you miss one, revisit that day before moving on.

Roadmap at a Glance

Week	Focus
Week 1	Dart Fundamentals - syntax, types, control flow, functions, collections, null safety
Week 2	Dart OOP & Advanced - classes, inheritance, mixins, generics, exceptions, async
Week 3	Flutter Basics - widgets, layouts, lists, theming
Week 4	Navigation, Forms & State - routing, validation, setState, Provider
Week 5	State Management + Networking - Provider/Riverpod/Bloc, REST, JSON
Week 6	Advanced Flutter - storage, Firebase, animations, testing, deployment
Week 7-8	Capstone Project (Days 43-56) + Interview Preparation (Days 57-60)

Part 1 · The 60-Day Plan, Day by Day

For each day: the topic, what to learn (explained), and a project to build with steps and tips.

WEEK 1: Dart Fundamentals

DAY 1

Introduction to Dart & Flutter + Setup

WHAT YOU'LL LEARN

- **What Flutter is:** Flutter is Google's open-source UI toolkit for building apps for mobile, web and desktop from a single codebase. It does not reuse the platform's native buttons or a webview; it paints every pixel itself, so your UI looks the same everywhere.
- **What Dart is:** Dart is the programming language Flutter uses. It has a clean, C-style syntax, is strongly typed, and compiles to fast native code for release builds while supporting instant hot reload during development.
- **Install the tooling:** Install the Flutter SDK (it bundles the Dart SDK), then an editor: VS Code or Android Studio, each with the official Flutter and Dart plugins. Finally set up an Android emulator, iOS simulator, or a real device.
- **flutter doctor:** Run `flutter doctor` in a terminal. It checks your whole setup and lists anything missing (Android licenses, Xcode, a connected device). Fix every red X before writing code.
- **The main() function:** Every Dart program starts at the `main()` function. It is the single entry point the runtime calls first.
- **Console input & output:** `print('hello')` writes a line to the console. `stdin.readLineSync()` reads one line the user types. These let you make interactive console programs before touching UI.

DAILY PROJECT · Welcome App (console)

WHAT YOU'LL BUILD

A tiny Dart console program that asks the user a few questions and prints a friendly, personalized greeting. The goal today is to prove your toolchain works end to end.

STEPS TO DO

1. Create a new Dart file and add a `main()` function.
2. Print a prompt, then read the user's name with `stdin.readLineSync()`.
3. Ask for their city and favourite hobby the same way.
4. Print a greeting that uses string interpolation, e.g. `"Hi $name from $city!"`.
5. Run it with `dart run` and confirm the output.

TIPS & SUGGESTIONS

- Import `dart:io` at the top so `stdin` is available.
- Don't jump into Flutter UI yet - today is purely about a working setup.
- If `flutter doctor` shows problems, fix them now; setup pain only gets worse later.

DAY 2

Variables, Data Types & Operators

WHAT YOU'LL LEARN

- **Declaring variables:** `var` lets Dart infer the type; `final` is set once at runtime and can't change; `const` is a compile-time constant; `late` promises you'll assign before first use. Prefer `final/const` for safety.
- **Built-in types:** The core types are `int`, `double` (and their shared parent `num`), `String`, `bool`, plus `dynamic` and `Object`. Dart is type-safe, so the compiler catches mismatches.

- **String interpolation:** Build strings with `$variable` or `${expression}` instead of clunky concatenation, e.g. `"Total: ${price * qty}"`.
- **Arithmetic operators:** `+` `-` `*` `/` plus `~/` (integer division) and `%` (remainder). Note `/` always returns a double.
- **Comparison & logical operators:** Compare with `==` `!=` `>` `<` `>=` `<=` and combine conditions with `&&` (and), `||` (or), `!` (not).
- **Null-aware operators:** `??` gives a fallback when a value is null, `??=` assigns only if currently null, and `?.` calls a member only if the object isn't null.
- **Type conversion:** Convert with helpers like `int.parse('5')`, `double.parse(...)`, `.toString()`, and check/cast types with `is` and `as`.

DAILY PROJECT · BMI Calculator (console)

WHAT YOU'LL BUILD

A program that reads a person's weight and height, computes their Body Mass Index, and prints a category (underweight, normal, overweight).

STEPS TO DO

1. Read weight (kg) and height (m) as strings, then parse to double.
2. Compute BMI = weight / (height * height).
3. Use comparisons to decide the category.
4. Print the BMI rounded to one decimal with `.toStringAsFixed(1)` and the category.

TIPS & SUGGESTIONS

- Guard against bad input - if parsing fails you'll get an error; you'll add proper handling on Day 13.
- Use `final` for values that never change after assignment.
- Practise interpolation in the output message rather than string '+' joins.

DAY 3

Control Flow: Conditional Statements

WHAT YOU'LL LEARN

- **if / else if / else:** Run different code based on conditions. Keep conditions readable and avoid deeply nested ifs where a switch is clearer.
- **The ternary operator:** `condition ? a : b` is a compact one-line if/else, great for choosing between two values.
- **switch statements:** Cleanly handle many discrete cases of one value. In modern Dart, `switch` can also be an expression that returns a value.
- **Pattern matching (Dart 3):** Switch supports patterns - match on shapes, types, and destructure values, making complex branching concise and exhaustive.
- **assert:** `assert(condition)` checks an assumption during development and is removed in release builds; useful for catching bugs early.

DAILY PROJECT · Grading System (console)

WHAT YOU'LL BUILD

A program that turns a numeric exam score into a letter grade and a short remark.

STEPS TO DO

1. Read the score and parse it to an `int`.
2. Use `if/else if` or a `switch` expression to map ranges to grades (A/B/C/D/F).
3. Print the grade plus a remark (e.g. 'Excellent', 'Needs work').
4. Reject scores outside 0-100 with a clear message.

TIPS & SUGGESTIONS

- Try writing it once with `if/else` and once with a `switch` expression to compare readability.
- Order your conditions from highest to lowest range to avoid logic gaps.
- Remember `/` gives a double - use `~/` if you ever need integer division here.

DAY 4

Loops: for, while, do-while

WHAT YOU'LL LEARN

- **The for loop:** Best when you know how many times to repeat: `for (var i = 0; i < n; i++)`. The classic counting loop.
- **for-in loop:** Iterate directly over the items of a list or other iterable: `for (final item in items)`. Cleaner when you don't need the index.
- **while loop:** Repeats as long as a condition stays true; the condition is checked before each pass, so it may run zero times.
- **do-while loop:** Like while, but the body runs at least once because the condition is checked at the end - ideal for menus.
- **break & continue:** `break` exits the loop immediately; `continue` skips to the next iteration. Use sparingly for clarity.

DAILY PROJECT · Number Guessing Game (console)

WHAT YOU'LL BUILD

The program picks a secret number and the player guesses it within a limited number of attempts, getting 'higher' or 'lower' hints.

STEPS TO DO

1. Pick a random target using `Random().nextInt(100)` from `dart:math`.
2. Loop up to N attempts, reading a guess each time.
3. Compare the guess and print 'too high' / 'too low' / 'correct'.
4. End early with `break` on a correct guess; report win or loss at the end.

TIPS & SUGGESTIONS

- A `while` loop with an attempts counter works nicely here.
- Validate that the guess parses to an `int` before comparing.
- Seed `Random()` only if you want reproducible runs for testing.

DAY 5

Functions

WHAT YOU'LL LEARN

- **Defining functions:** A function has a return type, a name, and parameters: `int add(int a, int b) { return a + b; }`. Use `void` when nothing is returned.
- **Positional vs named parameters:** Positional params depend on order; named params are passed by name in `{}` for readability, e.g. `box(width: 5, height: 2)`.
- **Optional & default values:** Named or `[]` positional params can be optional and have defaults; mark a named param `required` when it must be supplied.
- **Arrow functions:** For one-expression functions, use the fat arrow: `int square(int x) => x * x;`.
- **First-class functions & closures:** Functions are values - you can store them in variables, pass them as arguments, and a closure can capture variables from its surrounding scope.
- **Recursion:** A function can call itself to solve problems defined in terms of smaller versions (e.g., factorial). Always include a base case.

DAILY PROJECT · Console Calculator

WHAT YOU'LL BUILD

A menu-driven calculator that performs add, subtract, multiply and divide using separate functions.

STEPS TO DO

1. Write one function per operation, each returning a result.
2. Show a menu and read the user's choice and two numbers.
3. Call the matching function and print the result.
4. Loop so the user can do more calculations until they quit.

TIPS & SUGGESTIONS

- Keep each function small and single-purpose - it's your first taste of clean code.
- Use named parameters if it makes a call clearer.
- Return values from functions rather than printing inside them; print in `main()`.

DAY 6

Collections: List, Set & Map

WHAT YOU'LL LEARN

- **List:** An ordered, indexable collection: `List<String> names = ['Asha', 'Bina']`. Access by index, add with `.add()`, remove with `.remove()`.
- **Set:** An unordered collection of unique items - perfect for removing duplicates or membership checks with `.contains()`.
- **Map:** Key-value pairs: `Map<String, int> ages = {'Asha': 30}`. Look up by key; keys are unique.
- **Useful methods:** Transform and query collections with `map()`, filter with `where()`, combine with `reduce()/fold()`, and count with `length`.
- **Spread & collection-if/for:** Insert one collection into another with `...`, and build lists conditionally using `if/for` directly inside a literal.
- **Iterable vs List:** Many operations return a lazy `Iterable`; call `.toList()` to materialize it when you need a concrete list.

DAILY PROJECT · Word Analyzer (console)

WHAT YOU'LL BUILD

Takes a sentence and reports the longest word, the number of vowels, and the sentence reversed word-by-word.

STEPS TO DO

1. Split the sentence into words with `.split(' ')`.
2. Use `reduce()` to find the longest word.
3. Count vowels by filtering characters with `where()`.
4. Reverse the word list and join it back into a string.

TIPS & SUGGESTIONS

- Lowercase the text first so vowel counting is consistent.
- `split('')` turns a string into a list of characters.
- Chain methods (`split -> where -> length`) to write expressive, compact code.

DAY 7

Strings & Sound Null Safety

WHAT YOU'LL LEARN

- **String methods:** Manipulate text with `toUpperCase()`, `trim()`, `substring()`, `replaceAll()`, `split()`, and `contains()`.
- **StringBuffer:** When building large strings in a loop, use `StringBuffer` and `.write()` for efficiency instead of repeated concatenation.
- **Sound null safety:** By default a type cannot be null: a `String` always holds text. To allow null you must write `String?`. This catches null errors at compile time.
- **The ? and ! operators:** `String?` means 'may be null'. The bang `!` asserts a nullable value is not null right now (use carefully - it throws if you're wrong).
- **late & flow analysis:** `late` lets you declare a non-nullable variable now and assign it before first use. Dart's flow analysis also 'promotes' a nullable to non-null inside an `if (x != null)` block.

DAILY PROJECT · Contact Formatter (console)

WHAT YOU'LL BUILD

Builds a neat contact card from fields where some (phone, email) may be missing, handling nulls gracefully.

STEPS TO DO

1. Model a contact with nullable fields like `String? phone`.
2. Use `??` to substitute 'N/A' when a field is null.
3. Use a `StringBuffer` to assemble the card text.
4. Print the formatted card.

TIPS & SUGGESTIONS

- Reach for `??` instead of long `if (x == null)` blocks.
- Avoid `!` unless you're certain a value isn't null.
- Notice how null safety forces you to think about missing data up front.

WEEK 2: Dart OOP + Advanced Dart

DAY 8

Classes & Objects

WHAT YOU'LL LEARN

- **Classes and objects:** A class is a blueprint; an object is an instance created from it with `var p = Person()`. Classes bundle data (fields) and behavior (methods).
- **Fields and methods:** Fields store an object's data; methods are functions that belong to the class and can use those fields via `this`.
- **Getters and setters:** Expose computed or controlled access to data: `int get isAdult => age >= 18;` looks like a field but runs code.
- **Instance vs static:** Instance members belong to each object; `static` members belong to the class itself and are shared across all instances.
- **The cascade operator:** `..` lets you call several methods/setters on the same object in a row without repeating the variable name.

DAILY PROJECT · Bank Account class (console)

WHAT YOU'LL BUILD

A `BankAccount` class with `deposit` and `withdraw` methods that protects the balance from going negative.

STEPS TO DO

1. Create the class with a private `_balance` field.
2. Add `deposit(amount)` and `withdraw(amount)` methods with validation.
3. Expose balance through a getter only (no public setter).
4. Create an account in `main()` and run a few transactions.

TIPS & SUGGESTIONS

- Prefix private fields with `_` - that's how Dart marks library-private members.
- Return success/failure or print a message when a withdrawal is rejected.
- Keep validation logic inside the class, not in `main()`.

DAY 9

Constructors

WHAT YOU'LL LEARN

- **Default & parameterized constructors:** The constructor builds an object. Use the shorthand `Point(this.x, this.y);` to assign fields directly from parameters.
- **Named constructors:** Provide alternative ways to build an object, e.g. `Point.origin() : x = 0, y = 0;`. Great for clarity.
- **Initializer lists:** Run setup before the body using `: field = value, assert(...)`. Used to set final fields and validate inputs.
- **const constructors:** If all fields are final, a `const` constructor lets you create compile-time-constant, canonicalized objects - important for fast Flutter widgets.
- **factory constructors:** A `factory` can decide what to return: a cached instance, a subtype, or an object built from a `Map` (handy for `fromJson`).

DAILY PROJECT · Shape Factory (console)**WHAT YOU'LL BUILD**

A factory constructor that returns a `Circle` or `Square` based on a string, each able to report its area.

STEPS TO DO

1. Define a base `Shape` with an `area()` method.
2. Create `Circle` and `Square` subclasses.
3. Add a factory `Shape.create(String type, double size)` that returns the right one.
4. Build a few shapes and print their areas.

TIPS & SUGGESTIONS

- Use a named constructor for special cases like `Circle.unit()`.
- Factories are how you'll later parse JSON into models.
- Mark fields `final` and use `const` constructors where possible.

DAY 10**Inheritance & Mixins****WHAT YOU'LL LEARN**

- **extends & super:** A subclass extends a parent to reuse and specialize it. `super()` calls the parent's constructor; `super.method()` calls the parent's version.
- **Method overriding:** Redefine a parent method in a child and mark it `@override`. This is the basis of polymorphism.
- **Single inheritance:** A Dart class can extend only one parent, keeping the hierarchy simple and predictable.
- **Mixins:** A `mixin` lets you share methods across many unrelated classes using `with`, without classic inheritance - great for cross-cutting behavior.
- **Composition vs inheritance:** Often 'has-a' (composition) is more flexible than 'is-a' (inheritance). Prefer composition when classes don't share a true type relationship.

DAILY PROJECT · Employee Hierarchy (console)**WHAT YOU'LL BUILD**

An `Employee` base class with `Manager` and `Developer` subclasses that calculate pay differently, plus a reusable logging mixin.

STEPS TO DO

1. Create `Employee` with shared fields and a `calculatePay()` method.
2. Override `calculatePay()` in `Manager` (bonus) and `Developer` (overtime).
3. Write a `Logger` mixin with a `log()` method and apply it with `with`.
4. Store employees in a list and print each one's pay.

TIPS & SUGGESTIONS

- Use `@override` on every overridden method - it catches typos.
- Notice how one list of `Employee` can hold both subtypes (polymorphism).
- Keep the mixin focused on a single concern.

DAY 11**Abstract Classes, Interfaces & Sealed Classes****WHAT YOU'LL LEARN**

- **Abstract classes:** An `abstract` class can't be instantiated and may declare methods without bodies. Subclasses must implement those methods - it defines a contract plus shared code.
- **Implicit interfaces:** Every class defines an interface. Using `implements` forces you to provide your own version of every member (no inherited implementation).
- **extends vs implements:** `extends` inherits behavior; `implements` only adopts the contract. A class can implement many interfaces but extend only one class.
- **Sealed classes (Dart 3):** A `sealed` class restricts subtypes to the same file, so a `switch` over them can be exhaustively checked by the compiler.
- **Class modifiers:** Dart 3 adds `final`, `base`, `interface`, and `sealed` modifiers to control exactly how a class can be extended or implemented.

DAILY PROJECT · Payment Processor (console)

WHAT YOU'LL BUILD

A sealed `PaymentMethod` hierarchy (`Card`, `Wallet`, `Cash`) handled by one exhaustive `switch` that processes each differently.

STEPS TO DO

1. Define `sealed class PaymentMethod` and a few subclasses.
2. Write a `process(PaymentMethod m)` function using a `switch` on the type.
3. Let the compiler confirm you handled every case.
4. Process a list of mixed payments.

TIPS & SUGGESTIONS

- Try removing one case to see the compiler warn you - that's the power of `sealed`.
- Use patterns in the `switch` to destructure each subtype's data.
- Abstract classes are for shared code; sealed classes are for closed sets of types.

DAY 12

Enums, Generics & Extensions

WHAT YOU'LL LEARN

- **Enums:** An enum defines a fixed set of named values (e.g., `Status { active, paused, done }`). Enhanced enums can also hold fields and methods.
- **Generics:** Generics let a class or function work with any type safely: `class Box<T>` stores a `T`. This avoids duplication and keeps type safety.
- **Type constraints:** Restrict a generic with `extends`, e.g. `T extends num`, so you can only use it with numbers and call number methods.
- **Extension methods:** Add new methods to existing types without subclassing: `extension on String { String get capitalized => ... }`. Resolved at compile time.

DAILY PROJECT · Generic Repository (console)

WHAT YOU'LL BUILD

A reusable in-memory `Repository<T>` that can store, list and find any kind of item by id.

STEPS TO DO

1. Define class `Repository<T>` with an internal `List<T>`.
2. Add `add(item)`, `getAll()`, and `findById(id)` methods.
3. Create two models (e.g., `User`, `Product`) and reuse the same repository.
4. Add an extension method on one model for a formatted display string.

TIPS & SUGGESTIONS

- Generics are everywhere in Flutter (`List<Widget>`, `Future<T>`) - get comfortable now.
- Use an enhanced enum somewhere to represent a status field.
- Keep the repository type-agnostic - no hard-coded model logic inside it.

DAY 13

Exception Handling

WHAT YOU'LL LEARN

- **try / catch / finally:** Wrap risky code in `try`; handle errors in `catch`; run cleanup that must always happen in `finally`.
- **on for specific types:** Catch a particular exception type with `on FormatException catch (e)` to respond differently to different errors.
- **throw & rethrow:** Raise an error with `throw`; use `rethrow` inside a `catch` to log locally but let the caller also handle it.
- **Custom exceptions:** Create your own error types by implementing `Exception`, giving meaningful names like `InsufficientFundsException`.
- **Error vs Exception:** Exceptions are recoverable conditions you should handle; Errors (like a failed assert) usually indicate programming bugs you should fix.

DAILY PROJECT · Safe Calculator (console)

WHAT YOU'LL BUILD

Upgrade your calculator to handle divide-by-zero and invalid input using a custom exception, and log every caught error.

STEPS TO DO

1. Create a `DivideByZeroException` implements `Exception`.
2. Throw it in your divide function when the divisor is zero.
3. Wrap parsing and calculation in `try/catch` with `on` clauses.
4. Print a friendly message and log the error in `finally`.

TIPS & SUGGESTIONS

- Catch specific types before a generic catch.
- Never swallow errors silently - at minimum log them.
- A custom exception name documents what went wrong far better than a generic one.

DAY 14

Asynchronous Dart

WHAT YOU'LL LEARN

- **Future:** A `Future<T>` is a value that will arrive later (or fail). Network calls and file reads return Futures.
- **async & await:** Mark a function `async` and use `await` to pause until a Future completes - without blocking the app. The code reads top-to-bottom like sync code.
- **then & catchError:** An alternative callback style: `future.then((v) => ...).catchError((e) => ...)`. `async/await` is usually clearer.
- **Streams:** A `Stream<T>` delivers many values over time (e.g., live updates). Consume with `await for` or a generator using `async*` and `yield`.
- **The event loop:** Dart is single-threaded with an event loop. `await` schedules the rest of your function to run when the awaited work finishes, while other events keep flowing.
- **Isolates:** For heavy CPU work, run it in an isolate with `Isolate.run()` / `compute()` so the UI never freezes.

DAILY PROJECT · Fake API Fetcher (console)

WHAT YOU'LL BUILD

Simulate a network call that takes time and may fail, then handle both success and error paths.

STEPS TO DO

1. Write `Future<String> fetchUser()` that uses `Future.delayed(...)`.
2. Randomly throw an error to simulate a failed request.
3. Call it with `await` inside a try/catch in an `async main()`.
4. Bonus: build a `Stream<int>` countdown with `async*` and print each tick.

TIPS & SUGGESTIONS

- Remember `main()` can be `async` too.
- Awaiting inside try/catch is how you handle real network failures.
- This is the exact pattern you'll use for HTTP calls in Week 5.

WEEK 3: Flutter Basics & Widgets

DAY 15

Flutter Architecture & Your First App

WHAT YOU'LL LEARN

- **Everything is a widget:** In Flutter, UI is built by composing widgets - text, buttons, padding, even the app itself are widgets arranged in a tree.
- **runApp & the widget tree:** `runApp(MyApp())` puts your root widget on screen. Widgets nest inside widgets to form the tree that describes your UI.
- **MaterialApp & Scaffold:** `MaterialApp` sets up theming, routing and Material design. `Scaffold` gives a standard screen layout with an `AppBar`, `body`, and more.
- **Project structure:** Your code lives in `lib/`, dependencies and assets are declared in `pubspec.yaml`, and platform folders (`android/`, `ios/`) hold native config.
- **Hot reload vs hot restart:** Hot reload injects code changes while keeping app state - near-instant. Hot restart restarts the app and resets state.

DAILY PROJECT · Hello Flutter screen**WHAT YOU'LL BUILD**

Your first Flutter app: a styled welcome screen with an app bar, a centered headline, and a subtitle.

STEPS TO DO

1. Create a project with `flutter create hello_flutter`.
2. Replace `main.dart` with a `MaterialApp` + `Scaffold`.
3. Put a `Column` with two `Text` widgets in the body, centered.
4. Run on an emulator and try editing text + hot reload.

TIPS & SUGGESTIONS

- Use `const` on widgets that never change for better performance.
- Watch how hot reload keeps your place while updating the UI.
- Read the comments in the default counter app before deleting them.

DAY 16**Stateless vs Stateful Widgets****WHAT YOU'LL LEARN**

- **StatelessWidget:** Immutable UI that only depends on its inputs. If nothing changes internally, use this - it's simpler and cheaper.
- **StatefulWidget:** Has a companion `State` object holding data that can change over time, like a counter or form input.
- **setState:** Call `setState(() { ... })` to update state and tell Flutter to rebuild that widget with the new values.
- **BuildContext:** Passed into `build()`, it represents the widget's place in the tree and is used to look up things like the theme or to navigate.
- **Lifecycle methods:** `initState()` runs once at creation (set up controllers), `build()` describes the UI (may run often), and `dispose()` cleans up (always release controllers).

DAILY PROJECT · Tip Counter**WHAT YOU'LL BUILD**

A stateful screen with a number that you can increment, decrement and reset with buttons.

STEPS TO DO

1. Make a `StatefulWidget` with an `int _count` field.
2. Show `_count` in a large `Text` widget.
3. Add three buttons that call `setState` to change or reset it.
4. Prevent the count from going below zero.

TIPS & SUGGESTIONS

- Only the widget that calls `setState` rebuilds - keep state close to where it's used.
- Initialize state in the field declaration or `initState`, not in `build`.
- Extract the buttons row into its own widget to keep `build` tidy.

DAY 17**Core / Basic Widgets****WHAT YOU'LL LEARN**

- **Text & TextStyle:** Display text and style it via `TextStyle` (size, weight, color, spacing). `RichText`/`TextSpan` mix styles in one line.
- **Images & Icons:** `Image.asset` loads bundled images, `Image.network` loads from a URL, and `Icon` shows Material icons.
- **Buttons:** `ElevatedButton`, `TextButton`, `OutlinedButton` and `IconButton` each suit different emphasis; all take an `onPressed` callback.
- **Container:** A flexible box for padding, margin, color, rounded corners, borders and sizing via `decoration` and `constraints`.

DAILY PROJECT · Business Card UI

WHAT YOU'LL BUILD

A polished business-card screen with a circular avatar, name, role, and a row of contact icons/buttons.

STEPS TO DO

1. Use a `Container` with padding and a background color or gradient.
2. Add a `CircleAvatar` for the photo and styled `Text` for name/role.
3. Place contact actions (call, email) in a `Row` of `IconButton`s.
4. Add a divider and some spacing for a clean look.

TIPS & SUGGESTIONS

- Group related styles into reusable `TextStyle` constants.
- Use `SizedBox` for spacing rather than empty containers.
- Add a subtle `BoxShadow` in the decoration for depth.

DAY 18

Layout Widgets

WHAT YOU'LL LEARN

- **Row & Column:** Arrange children horizontally (`Row`) or vertically (`Column`). Control spacing with `mainAxisAlignment` and `crossAxisAlignment`.
- **Expanded & Flexible:** Inside a `Row`/`Column`, `Expanded` makes a child fill remaining space; `Flexible` lets it take up to its share. Great for proportional layouts.
- **Stack & Positioned:** `Stack` layers widgets on top of each other; `Positioned` places a child precisely - used for badges, overlays, banners.
- **Spacing & alignment:** `Padding`, `Center`, `Align`, and `SizedBox` fine-tune position and whitespace.
- **The constraint model:** Remember the rule: constraints flow down, sizes flow up, and the parent sets the position. Understanding this prevents most layout errors.

DAILY PROJECT · Dashboard Layout

WHAT YOU'LL BUILD

A responsive dashboard with a header and a grid of statistic cards using Rows, Columns and Expanded.

STEPS TO DO

1. Build a header row with a title and an icon.
2. Create a reusable StatCard widget.
3. Lay the cards out in Rows with Expanded so they share width evenly.
4. Add padding and spacing so it looks balanced.

TIPS & SUGGESTIONS

- Wrap children in Expanded to avoid overflow on small screens.
- If you see a yellow/black overflow stripe, a child is too big for its space.
- Extract repeated UI (the card) into its own widget.

DAY 19

Lists, Grids & Scrolling

WHAT YOU'LL LEARN

- **ListView:** Displays a scrollable column of widgets. The default constructor builds all children at once - fine for short lists.
- **ListView.builder:** Builds items lazily as they scroll into view - essential for long or infinite lists. Provide `itemCount` and `itemBuilder`.
- **GridView:** Lays items out in a scrollable grid; `GridView.builder` with a `SliverGridDelegate` controls columns and spacing.
- **ListTile & Card:** `ListTile` is a ready-made row (leading icon, title, subtitle, trailing). `Card` adds elevation and rounded corners.
- **Scrolling helpers:** `SingleChildScrollView` makes any column scrollable; a `ScrollController` lets you detect position for things like infinite scroll.

DAILY PROJECT · Recipe List

WHAT YOU'LL BUILD

A scrollable list of recipe cards built from a Dart list, each showing an image, title and short description.

STEPS TO DO

1. Create a List of simple recipe maps or model objects.
2. Use `ListView.builder` to render one Card per recipe.
3. Inside each card use a `ListTile` or a custom Row layout.
4. Tap a card to print the recipe name (real navigation comes Day 22).

TIPS & SUGGESTIONS

- Always prefer `.builder` for lists of unknown or large length.
- Give list items a key if they can be reordered (you'll learn why later).
- Use `Image.network` with a `loadingBuilder` for nicer image loading.

DAY 20

Styling, Theming & Assets

WHAT YOU'LL LEARN

- **ThemeData & Material 3:** Define your app's colors, text styles and shapes once in `ThemeData` (with `useMaterial3: true`) so the whole app stays consistent.
- **Color schemes & typography:** Use a `ColorScheme.fromSeed(...)` to generate a harmonious palette, and a `TextTheme` for consistent text styles.
- **Light & dark themes:** Provide both `theme` and `darkTheme`; Flutter switches based on the system or your own toggle.
- **Adding assets & fonts:** Register images and custom fonts in `pubspec.yaml` so they're bundled and usable via `Image.asset` and a `FontFamily`.
- **Responsiveness:** `MediaQuery` gives screen size/orientation; `LayoutBuilder` gives the parent's constraints so you can adapt layouts.

DAILY PROJECT · Themed Quotes App

WHAT YOU'LL BUILD

A list of styled quote cards with a working light/dark mode toggle and a custom font.

STEPS TO DO

1. Define a seed-based `ColorScheme` and a `darkTheme`.
2. Add a custom font in `pubspec.yaml` and use it in your `TextTheme`.
3. Show quotes in styled cards.
4. Add a switch in the app bar to toggle theme mode.

TIPS & SUGGESTIONS

- Read colors from `Theme.of(context).colorScheme` instead of hard-coding them.
- Run the app in both light and dark to check contrast.
- Keep all assets under an `assets/` folder and list each one in `pubspec`.

DAY 21

Week 3 Review & Mini Project

WHAT YOU'LL LEARN

- **Composition:** Build complex screens by composing small, focused widgets. Extract anything you repeat into its own widget.
- **Widget extraction:** Pull big `build` methods apart into separate widget classes for readability, reuse and performance.
- **Consistency:** Reuse your theme, spacing and text styles so the whole screen feels cohesive.

DAILY PROJECT · Profile / Portfolio Screen**WHAT YOU'LL BUILD**

A single polished screen with a header (avatar, name, tagline), an About section, a row of skill chips, and a list of projects.

STEPS TO DO

1. Lay out the header with a `Column` inside a styled `Container`.
2. Add an About paragraph and a `Wrap` of `Chip` widgets for skills.
3. List projects with `ListView` or a `Column` of cards.
4. Apply your theme consistently across all sections.

TIPS & SUGGESTIONS

- Break the screen into `_Header`, `_About`, `_Skills`, `_Projects` widgets.
- Use `Wrap` so skill chips flow onto new lines automatically.
- This screen is a great first piece for your real portfolio - make it nice.

WEEK 4: Navigation, Forms & State**DAY 22****Navigation Basics****WHAT YOU'LL LEARN**

- **The Navigator stack:** Flutter manages screens as a stack of routes. You push a new screen on top and pop to go back, like a stack of cards.
- **push & MaterialPageRoute:** `Navigator.push(context, MaterialPageRoute(builder: (_) => DetailScreen()))` opens a new screen with a platform-appropriate transition.
- **Passing data forward:** Send data to the next screen through its constructor: `DetailScreen(product: p)`. Simple and type-safe.
- **Returning data back:** `pop(result)` returns a value to the screen that pushed; `await` the push to receive it (e.g., a selected item).
- **Handling back:** `PopScope` (formerly `WillPopScope`) lets you intercept the back action, e.g., to confirm before discarding changes.

DAILY PROJECT · Two-Screen App**WHAT YOU'LL BUILD**

A product list where tapping an item opens a detail screen, and the detail screen can send a result (like 'added to cart') back.

STEPS TO DO

1. Build a list screen and a detail screen.
2. On tap, push the detail screen, passing the selected product.
3. On the detail screen, pop a result when the user taps a button.
4. `await` the push on the list screen and show the returned result.

TIPS & SUGGESTIONS

- Type your route results, e.g. `await Navigator.push<bool>(...)`.
- Pass whole model objects via the constructor rather than loose fields.
- Keep each screen in its own file as your app grows.

DAY 23

Named Routes & go_router

WHAT YOU'LL LEARN

- **Named routes:** Register routes by name in `MaterialApp(routes: {...})` and navigate with `Navigator.pushNamed(context, '/settings')` - cleaner for many screens.
- **Passing arguments:** Send data to a named route via the `arguments` parameter and read it with `ModalRoute.of(context)!.settings.arguments`.
- **onGenerateRoute:** For dynamic routes (like `/product/42`) implement `onGenerateRoute` to parse the route and build the right screen.
- **go_router:** The recommended package for larger apps: declarative, URL-based routing that handles deep links and the web address bar cleanly.
- **Nested & shell routes:** `go_router` supports nested navigation (e.g., a bottom nav bar that keeps its own stack per tab).

DAILY PROJECT · Multi-Page App

WHAT YOU'LL BUILD

A small app with Home, Settings and About pages reachable via a navigation drawer, first with named routes then refactored to `go_router`.

STEPS TO DO

1. Create three screens and register named routes.
2. Add a `Drawer` with links that navigate to each.
3. Pass a simple argument to one screen.
4. Refactor the routing to use `go_router` and compare the code.

TIPS & SUGGESTIONS

- Start with named routes to understand the basics, then appreciate what `go_router` simplifies.
- `go_router` is especially worth it if you target web.
- Centralize your route definitions in one file.

DAY 24

Forms & Validation

WHAT YOU'LL LEARN

- **Form & FormState:** Wrap fields in a `Form` and hold a `GlobalKey<FormState>` to validate and save them together.
- **TextFormField:** A text input with built-in validation support. Provide a `validator` that returns an error string or null when valid.
- **Controllers:** A `TextEditingController` reads and sets a field's text; remember to `dispose()` it in a `StatefulWidget`.
- **Validation flow:** Call `_formKey.currentState!.validate()` on submit; show errors automatically. `autovalidateMode` can validate as the user types.
- **Focus management:** `FocusNode` lets you move focus between fields (e.g., 'next' on the keyboard).

DAILY PROJECT · Sign-Up Form

WHAT YOU'LL BUILD

A registration form with name, email and password fields, full validation, and a success message on submit.

STEPS TO DO

1. Create a Form with three `TextFormField`s and controllers.
2. Write validators (non-empty name, email pattern, min password length).
3. On submit, validate; if valid, show a `SnackBar` confirmation.
4. Dispose controllers in `dispose()`.

TIPS & SUGGESTIONS

- Use `keyboardType` and `obscureText` for email and password fields.
- Validate on submit first; add live validation only if it helps UX.
- Trim input before validating to avoid whitespace bugs.

DAY 25

Gestures & Interaction

WHAT YOU'LL LEARN

- **GestureDetector:** Wrap any widget to detect taps, double-taps, long-presses, drags and more via callbacks like `onTap`.
- **InkWell:** Like `GestureDetector` but adds a Material ripple effect - use it for tappable Material surfaces.
- **Dismissible:** Swipe-to-dismiss list items with a background reveal; great for delete/archive actions.
- **Dialogs & sheets:** Show modal content with `showDialog`, `showModalBottomSheet`, and quick feedback with `SnackBar`.

DAILY PROJECT · Swipe To-Do

WHAT YOU'LL BUILD

A task list where tapping a task toggles 'done' and swiping it away deletes it, with a confirmation `SnackBar`.

STEPS TO DO

1. Render tasks with `ListView.builder`.
2. Wrap each item in `Dismissible` with a red delete background.
3. Toggle completion on tap (strike-through the text).
4. Show an undo `SnackBar` after deleting.

TIPS & SUGGESTIONS

- Give each `Dismissible` a unique key.
- Update the underlying list, then `setState` to reflect changes.
- Use `InkWell` for the tap so users get visual feedback.

DAY 26

Managing State with `setState`

WHAT YOU'LL LEARN

- **Local state:** For state used by a single widget, `setState` is the right, simplest tool. Don't over-engineer small features.
- **Lifting state up:** When two widgets need the same data, move that state to their nearest common ancestor and pass it down.

- **Passing data down & callbacks up:** Send data to children via constructors; let children report events back via callback functions you pass in.
- **Limits of setState:** When state must be shared across many distant widgets, setState gets messy - that's the signal to adopt a state management solution.

DAILY PROJECT · Shopping Cart (setState)

WHAT YOU'LL BUILD

A product list and a cart total that update together: adding/removing items recalculates the total, with state lifted to a parent.

STEPS TO DO

1. Hold the cart list in a parent widget.
2. Pass an `onAdd/onRemove` callback down to the product list.
3. Show a running total in a header that updates via `setState`.
4. Disable 'add' once an item is in the cart, if you like.

TIPS & SUGGESTIONS

- Notice how lifting state keeps the list and total in sync.
- Keep child widgets 'dumb' - they display data and call callbacks.
- Feel the pain points; tomorrow's Provider solves them.

DAY 27

InheritedWidget & Provider Intro

WHAT YOU'LL LEARN

- **Why state management:** Passing data through many widget layers ('prop drilling') is tedious. State management shares data cleanly across the tree.
- **InheritedWidget:** Flutter's built-in mechanism for efficiently passing data down the tree; it's what `Theme.of(context)` uses under the hood.
- **ChangeNotifier:** A simple class that holds state and calls `notifyListeners()` when it changes, telling the UI to rebuild.
- **Provider package:** `ChangeNotifierProvider` exposes a notifier to the tree; widgets read it with `context.watch/read` or a `Consumer`.

DAILY PROJECT · Counter with Provider

WHAT YOU'LL BUILD

A shared counter that two different screens can read and update through a single `ChangeNotifier`.

STEPS TO DO

1. Add the provider package to `pubspec.yaml`.
2. Create a `CounterModel` extends `ChangeNotifier` with `increment()`.
3. Wrap the app in `ChangeNotifierProvider`.
4. Read/update the counter from two screens with `context.watch/read`.

TIPS & SUGGESTIONS

- `context.watch` rebuilds on change; `context.read` is for one-off calls in callbacks.
- Always call `notifyListeners()` after changing state.
- Notice you no longer pass the counter through constructors.

DAY 28

Week 4 Review & Mini Project

WHAT YOU'LL LEARN

- **Putting it together:** Combine navigation, forms, validation and Provider into one coherent multi-screen app.
- **Separation of concerns:** Keep UI in widgets and shared state in a notifier - your screens stay simple and testable.
- **Clean structure:** Organize files by feature (screens, models, providers) so the project scales.

DAILY PROJECT · To-Do App (multi-screen)

WHAT YOU'LL BUILD

A multi-screen task manager: a list screen, an add/edit form screen, with tasks stored in a Provider so both screens stay in sync.

STEPS TO DO

1. Model a `Task` and a `TaskProvider` extends `ChangeNotifier`.
2. List screen reads tasks; an 'add' button navigates to a form.
3. The form validates input and adds/edits a task in the provider.
4. Support deleting and marking tasks done.

TIPS & SUGGESTIONS

- Reuse the same form screen for both add and edit by passing an optional task.
- Keep all task logic in the provider, not the widgets.
- Commit this to GitHub - it's a solid portfolio starter.

WEEK 5: State Management + Networking

DAY 29

Provider Deep Dive

WHAT YOU'LL LEARN

- **MultiProvider:** Register several providers at once with `MultiProvider` when your app has more than one piece of shared state.
- **watch / read / select:** `watch` listens and rebuilds, `read` accesses without listening, and `select` listens to just one field to minimize rebuilds.
- **Consumer vs Selector:** `Consumer` rebuilds when the notifier changes; `Selector` rebuilds only when a chosen value changes - better for performance.
- **Provider variants:** `FutureProvider` and `StreamProvider` expose async values, and `ProxyProvider` builds one provider from another.

DAILY PROJECT · Cart + Theme (MultiProvider)**WHAT YOU'LL BUILD**

An app with two independent providers - a shopping cart and a theme toggle - wired together with MultiProvider and optimized with Selector.

STEPS TO DO

1. Create a CartProvider and a ThemeProvider.
2. Register both via MultiProvider.
3. Use Selector so the cart badge rebuilds only when the count changes.
4. Toggle theme from anywhere using context.read.

TIPS & SUGGESTIONS

- Reach for Selector/select when you notice unnecessary rebuilds.
- Keep providers small and focused on one concern.
- Use the Flutter DevTools rebuild counter to verify your optimization.

DAY 30**Riverpod & Bloc Overview****WHAT YOU'LL LEARN**

- **Riverpod:** A modern, compile-safe evolution of Provider. You read providers via a ref without needing BuildContext, and it's very testable.
- **Riverpod notifiers:** Notifier/AsyncNotifier hold and expose state; AsyncValue neatly models loading/data/error in one type.
- **Bloc / Cubit:** The Bloc pattern separates UI from logic via events and states. A Cubit is a lighter version that exposes methods instead of events.
- **Choosing an approach:** setState for local state, Provider/Riverpod for most apps, Bloc for large teams needing strict, predictable, event-driven flows.

DAILY PROJECT · Counter x3**WHAT YOU'LL BUILD**

Implement the same simple counter feature three ways - setState, Riverpod, and Cubit - to feel the trade-offs.

STEPS TO DO

1. Build the counter once with setState.
2. Rebuild it with a Riverpod Notifier and ConsumerWidget.
3. Rebuild it with a Cubit and BlocBuilder.
4. Write a short note on what you preferred and why.

TIPS & SUGGESTIONS

- Don't memorize all three - understand when each shines.
- Riverpod's no-context reads are great for testing and services.
- Pick one main solution for your capstone and go deep on it.

DAY 31**HTTP & REST APIs****WHAT YOU'LL LEARN**

- **REST basics:** REST APIs expose resources over HTTP using verbs: GET (read), POST (create), PUT/PATCH (update), DELETE (remove).
- **The http package:** Use `http.get(Uri.parse(url))` to call an endpoint and read `response.statusCode` and `response.body`.
- **Headers & body:** Send headers (like auth tokens and content-type) and a JSON body for POST/PUT requests.
- **Status codes:** 2xx means success, 4xx is a client error (bad request, unauthorized, not found), 5xx is a server error. Check them before parsing.
- **dio:** A popular alternative client with interceptors, base URLs, timeouts and easier error handling for bigger apps.

DAILY PROJECT · Quote Fetcher

WHAT YOU'LL BUILD

An app that fetches a random quote from a public API and displays it, with a button to fetch another.

STEPS TO DO

1. Add the `http` package.
2. Call a public quotes API with `http.get`.
3. Check the status code, then decode the JSON.
4. Show the quote and a 'New quote' button.

TIPS & SUGGESTIONS

- Always check `statusCode` before using the body.
- Wrap network calls in try/catch (you'll formalize this on Day 34).
- Use a free public API like a quotes or JSONPlaceholder endpoint to start.

DAY 32

JSON Serialization

WHAT YOU'LL LEARN

- **Encode & decode:** `jsonDecode(string)` turns JSON text into Dart Maps/Lists; `jsonEncode(object)` does the reverse.
- **Models with fromJson/toJson:** Convert raw maps into typed model classes via a factory `User.fromJson(Map json)` and a `toJson()` method - safer and clearer than using maps directly.
- **Nested objects & lists:** Parse nested JSON by calling each sub-model's `fromJson`, and map JSON arrays into `List<Model>`.
- **Code generation:** For big models, `json_serializable` (with `build_runner`) or `freezed` generate the boilerplate for you, including immutability and `copyWith`.

DAILY PROJECT · User Directory**WHAT YOU'LL BUILD**

Fetch a list of users from an API, parse the JSON into model objects, and display them in a list.

STEPS TO DO

1. Define a `User` model with `fromJson`.
2. Fetch the user list and decode the JSON array.
3. Map the decoded list into `List<User>`.
4. Render names and emails in a `ListView`.

TIPS & SUGGESTIONS

- Handle missing/nullable JSON fields with `?` and defaults.
- Keep models in their own files for reuse.
- Try generating one model with `json_serializable` to see the time saved.

DAY 33**FutureBuilder & StreamBuilder****WHAT YOU'LL LEARN**

- **FutureBuilder:** Rebuilds your UI based on a single `Future`'s progress - perfect for showing a spinner then the loaded data.
- **AsyncSnapshot:** The builder gives you a snapshot with `connectionState`, `hasData`, `data`, and `error` - use these to show loading/data/error UI.
- **StreamBuilder:** Like `FutureBuilder` but for `Streams` - rebuilds on every emitted event (e.g., a live chat or a timer).
- **Common pitfall:** Never create the `Future/Stream` inside `build()`, or it re-runs on every rebuild. Create it in `initState` or a provider.

DAILY PROJECT · GitHub User Search**WHAT YOU'LL BUILD**

Search a username, call the API, and display the profile with proper loading, error and empty states using `FutureBuilder`.

STEPS TO DO

1. Add a search field and store the query.
2. Trigger a `Future` that fetches the user.
3. Use `FutureBuilder` to show a spinner, the result, or an error.
4. Handle 'not found' as a friendly empty state.

TIPS & SUGGESTIONS

- Store the `Future` in state so it isn't recreated on each rebuild.
- Always handle `snapshot.hasError`, not just success.
- Debounce the search if you call the API as the user types.

DAY 34**Architecture, Errors & Loading States****WHAT YOU'LL LEARN**

- **Repository pattern:** Put data-fetching logic behind a repository so your UI/state layer doesn't talk to `http` directly. Easier to test and swap.

- **Service & layers:** Separate concerns: data layer (API/db), domain/state layer (logic), and UI layer (widgets).
- **Centralized error handling:** Catch and translate errors into meaningful app states or messages in one place, not scattered through the UI.
- **UI states:** Model every async screen as loading, data, empty, or error - and design the UI for each, including a retry action.

DAILY PROJECT · Clean Posts App

WHAT YOU'LL BUILD

Refactor an API feature into a repository + model + provider with proper loading/error/empty states.

STEPS TO DO

1. Create a `PostRepository` that returns parsed models.
2. Expose state from a provider (loading/data/error).
3. Build UI that reacts to each state.
4. Add a retry button on error.

TIPS & SUGGESTIONS

- Mock the repository later to unit-test your provider.
- A simple enum or sealed class is great for representing UI state.
- Show empty states ('No posts yet') - they're part of good UX.

DAY 35

Week 5 Review & API Project

WHAT YOU'LL LEARN

- **End-to-end flow:** Combine networking, models, state management and polished states into one real app.
- **Robustness:** Handle slow networks, failures and empty results gracefully.
- **Polish:** Loading spinners, error messages and retry turn a demo into something usable.

DAILY PROJECT · Weather or News App

WHAT YOU'LL BUILD

A complete app where the user searches a city or topic and sees results fetched from a REST API, with clean loading and error handling.

STEPS TO DO

1. Pick a free API (weather or news) and read its docs.
2. Build models, a repository, and a provider.
3. Create a search UI and a results list/detail.
4. Handle loading, errors, and empty results everywhere.

TIPS & SUGGESTIONS

- Keep your API key out of source control (use a config or env approach).
- Cache the last result so the screen isn't blank on reopen (optional).
- This is a strong portfolio piece - make the UI clean.

WEEK 6: Advanced Flutter + Real Projects

DAY 36

Local Persistence & Storage

WHAT YOU'LL LEARN

- **shared_preferences:** Stores small key-value data (settings, flags, a saved token) that survives app restarts. Not for large or relational data.
- **sqlite:** A SQLite database for structured, relational local data with full SQL queries - good for offline records.
- **Hive / Isar:** Fast NoSQL/object databases that store Dart objects directly - excellent for offline-first apps and simple to use.
- **Files & path_provider:** Use `path_provider` to find safe directories, then read/write files for documents, cache, or exports.

DAILY PROJECT · Notes App (offline)

WHAT YOU'LL BUILD

A notes app where users create, read, update and delete notes stored locally so they persist between launches.

STEPS TO DO

1. Choose Hive (simplest) or sqlite.
2. Define a note model and initialize the database.
3. Implement create/read/update/delete.
4. Load notes on startup and reflect changes in the UI.

TIPS & SUGGESTIONS

- Initialize your storage in `main()` before `runApp`.
- Keep DB code in a repository, separate from widgets.
- Test that data survives a full app restart.

DAY 37

Firebase Setup & Authentication

WHAT YOU'LL LEARN

- **Firebase & FlutterFire:** Firebase is Google's backend platform (auth, database, storage). The FlutterFire CLI wires it into your app and generates config.
- **firebase_core:** Initialize Firebase once at startup with `Firebase.initializeApp()` before using any Firebase service.
- **Firebase Auth:** Register and sign in users with email/password or providers like Google. It handles tokens and sessions for you.
- **Auth state:** Listen to `authStateChanges()` to know if a user is signed in and route them to the right screen automatically.

DAILY PROJECT · Auth Flow

WHAT YOU'LL BUILD

A real authentication flow: register, log in, log out, and an auth-guarded home screen only signed-in users can see.

STEPS TO DO

1. Set up Firebase with the FlutterFire CLI.
2. Build register and login screens using Firebase Auth.
3. Listen to auth state to decide which screen to show.
4. Add a logout button.

TIPS & SUGGESTIONS

- Show clear error messages for wrong password / existing email.
- Use a `StreamBuilder` on `authStateChanges()` as your app's gate.
- Never store passwords yourself - let Firebase handle it.

DAY 38

Cloud Firestore

WHAT YOU'LL LEARN

- **Collections & documents:** Firestore stores data as documents (maps) inside collections - a flexible NoSQL structure.
- **CRUD operations:** Create with `add/set`, read with `get`, update with `update`, and delete with `delete` on document references.
- **Real-time streams:** Listen to a collection or document with `.snapshots()` to get live updates pushed to your UI automatically.
- **Queries & rules:** Filter and order with `where/orderBy`, paginate with `limit/startAfter`, and protect data with security rules.

DAILY PROJECT · Realtime Notes / Chat

WHAT YOU'LL BUILD

A Firestore-backed list (notes or chat messages) that updates live across devices in real time.

STEPS TO DO

1. Connect Firestore and define a collection.
2. Add documents from a form.
3. Display them with a `StreamBuilder` on `.snapshots()`.
4. Support editing and deleting documents.

TIPS & SUGGESTIONS

- Tie documents to the signed-in user's id for per-user data.
- Order by a timestamp so new items appear correctly.
- Set basic security rules - don't leave the database open.

DAY 39

Animations

WHAT YOU'LL LEARN

- **Implicit animations:** Widgets like `AnimatedContainer`, `AnimatedOpacity` and `AnimatedSwitcher` animate automatically when their properties change - simple and declarative.
- **Explicit animations:** Use an `AnimationController` plus a `Tween` for precise control over duration, curves, direction and replay.
- **Curves & vsync:** Apply easing with `CurvedAnimation`, and provide a `vsync` via `TickerProviderStateMixin` so animations stay smooth and battery-friendly.
- **Hero & transitions:** `Hero` animates a shared element between screens; you can also customize page route transitions.
- **Disposing controllers:** Always `dispose()` animation controllers to avoid memory leaks.

DAILY PROJECT · Animated Onboarding

WHAT YOU'LL BUILD

A multi-page onboarding flow with smooth page transitions, an animated illustration, and a `Hero` image into the main app.

STEPS TO DO

1. Build pages with `PageView` and animated indicators.
2. Use an implicit animation for a fading/scaling element.
3. Add one explicit `AnimationController`-driven animation.
4. Use `Hero` to transition an image into the home screen.

TIPS & SUGGESTIONS

- Start with implicit animations - they cover most needs.
- Keep durations short (200-400ms) for a snappy feel.
- Remember with `TickerProviderStateMixin` and to dispose controllers.

DAY 40

Testing

WHAT YOU'LL LEARN

- **Unit tests:** Test pure Dart logic (functions, models, services) in isolation with `test()` and `expect()`.
- **Widget tests:** Use `testWidgets`, `pumpWidget`, finders (`find.text`) and matchers (`findsOneWidget`) to verify UI behavior fast, without a device.
- **Mocking:** Replace real dependencies (like a repository) with mocks using `mocktail/mockito` so tests are fast and deterministic.
- **Integration tests:** The `integration_test` package runs the full app on a device/emulator to test real end-to-end flows.

DAILY PROJECT · Tested Counter + API**WHAT YOU'LL BUILD**

Add a unit test for a service and a widget test for a screen, mocking the network dependency.

STEPS TO DO

1. Write a unit test for a small piece of logic (e.g., a calculator or formatter).
2. Mock a repository and unit-test a provider's states.
3. Write a widget test that pumps a screen and checks what renders.
4. Run `flutter test` and make everything green.

TIPS & SUGGESTIONS

- Aim to test logic and critical UI, not every pixel.
- Inject dependencies so they're easy to mock.
- Tests give you confidence to refactor your capstone safely.

DAY 41**Performance & Deployment****WHAT YOU'LL LEARN**

- **const & rebuilds:** Mark unchanging widgets `const` and keep state localized so Flutter rebuilds as little as possible.
- **DevTools:** Use Flutter DevTools to inspect the widget tree, count rebuilds, and profile jank on the UI/raster threads.
- **Build artifacts:** Produce releasable builds: `flutter build appbundle` (Android), `flutter build ipa` (iOS), `flutter build web`.
- **Flavors & config:** Use flavors/environments to separate dev/staging/prod settings and keep secrets out of code.
- **CI/CD:** Automate building, testing and releasing with tools like GitHub Actions or Codemagic.

DAILY PROJECT · Release Build**WHAT YOU'LL BUILD**

Profile a previous project for unnecessary rebuilds, add `const` where possible, then produce a release artifact.

STEPS TO DO

1. Open DevTools and check rebuild counts on a busy screen.
2. Add `const` and split widgets to cut rebuilds.
3. Create a release build (app bundle or web).
4. Note the app size and startup time.

TIPS & SUGGESTIONS

- `const` is the cheapest performance win you can make.
- Test the release build on a real device, not just debug.
- Keep API keys and secrets out of version control.

DAY 42**Capstone Planning****WHAT YOU'LL LEARN**

- **Choosing a domain:** Pick something you can scope in two weeks and would be proud to show: E-Commerce, Social, Fitness, Finance, or an LMS.

- **Feature planning:** List must-have features (MVP) vs nice-to-haves. Build the MVP first.
- **Architecture decisions:** Decide your state management, navigation, data sources and folder structure up front.
- **Repo setup:** Initialize Git, add a README, set up your folder structure, and make the first commit.

DAILY PROJECT · Capstone Blueprint

WHAT YOU'LL BUILD

A written plan and skeleton for your capstone: chosen domain, feature list, data model, architecture, wireframes, and an initialized repo.

STEPS TO DO

1. Write a one-page spec: problem, features, screens.
2. Sketch wireframes (paper or a tool) for the main screens.
3. Define your data model and choose your stack.
4. Create the repo, folder structure, and initial commit.

TIPS & SUGGESTIONS

- Keep the MVP small - you can always add features later.
- Reuse patterns you've practised; don't invent new ones now.
- A clear plan makes the next two weeks far smoother.

WEEK 7-8: Capstone Project + Interview Prep

DAY 43-56

Build Your Full Flutter App (2 weeks)

WHAT YOU'LL LEARN

- **Scope and stack:** Build a complete app in your chosen domain using everything you've learned: navigation, forms, state management, networking and/or Firebase, local storage, animations and tests.
- **Core features first:** Implement the MVP end-to-end before polishing: authentication, the main CRUD flow, and the key screens.
- **Clean architecture:** Separate UI, state/logic and data (repositories). This keeps the app testable and easy to extend.
- **Daily commits:** Commit working code every day with clear messages. Small, frequent commits make your progress (and debugging) much easier.
- **Tests as you go:** Write unit and widget tests for important logic and screens while you build, not at the very end.
- **Polish & states:** Handle loading, empty and error states everywhere, add a consistent theme, and refine spacing and transitions.

DAILY PROJECT · Capstone App (CRM / E-Commerce / Fitness / LMS)**WHAT YOU'LL BUILD**

A portfolio-grade, full-stack Flutter app with authentication, a main feature set (CRUD), networking or Firebase, local caching, polished UI, and tests - pushed to GitHub.

STEPS TO DO

1. Days 43-45: scaffold the app, set up routing, theme, auth.
2. Days 46-50: build the core feature(s) end-to-end with state management and data.
3. Days 51-53: add secondary features (search, profile, settings, file upload).
4. Days 54-55: write tests and fix bugs.
5. Day 56: polish UI, handle all states, and finalize the README.

TIPS & SUGGESTIONS

- Resist scope creep - finish the MVP before adding extras.
- Reuse the patterns you practised; consistency beats cleverness.
- Record short demo GIFs as you finish features for your README.
- Push daily; a green commit history is great for recruiters.

DAY 57-58**Interview Preparation****WHAT YOU'LL LEARN**

- **Dart & OOP:** Review null safety, `final` vs `const`, Futures/Streams, isolates, mixins, abstract vs interface, and the SOLID principles.
- **Flutter internals:** Be ready to explain the widget/element/render trees, `BuildContext`, keys, the widget lifecycle, and the constraint model.
- **State management:** Compare `setState`, `Provider`, `Riverpod` and `Bloc`, and justify your capstone's choice.
- **Data structures & algorithms:** Practise common DSA problems in Dart (arrays, strings, maps, recursion) and explain your reasoning out loud.
- **App/system design:** Be able to outline how you'd structure an app: layers, data flow, navigation, offline support and error handling.

DAILY PROJECT · Interview Reps**WHAT YOU'LL BUILD**

Active practice: solve and explain technical questions, review the Q&A in this guide, and rehearse explaining your capstone's architecture.

STEPS TO DO

1. Work through the 45 interview questions in Part 2 - answer aloud first, then check.
2. Solve 1-2 DSA problems per day in Dart and explain the approach.
3. Draw your capstone's architecture and practise narrating it.
4. Take the quizzes in Part 3 and revisit any weak topics.

TIPS & SUGGESTIONS

- Explaining out loud is the real skill - record yourself.
- Connect every concept back to where you used it in your projects.
- It's fine to say 'I'd look this up' - reasoning matters more than recall.

DAY 59

Mock Interviews & Portfolio Polish

WHAT YOU'LL LEARN

- **Mock interviews:** Simulate real interviews: a technical round and a behavioral round. Practise structure (e.g., STAR for behavioral answers).
- **Project walkthrough:** Prepare a confident 2-3 minute tour of your capstone: the problem, the architecture, the hard parts and what you'd improve.
- **README & demo:** A great README with screenshots, a feature list, setup steps and a demo GIF/video makes your repo stand out.
- **Portfolio:** Pin your best repos, ensure they run, and write clear project descriptions.

DAILY PROJECT · Portfolio Polish

WHAT YOU'LL BUILD

Finalize a strong README for your capstone (screenshots, GIFs, architecture notes), record a short demo, and rehearse your walkthrough.

STEPS TO DO

1. Write a README: overview, features, tech stack, setup, screenshots.
2. Record a 60-second screen demo of the app.
3. Do one full mock technical and behavioral interview.
4. Pin and tidy your top GitHub projects.

TIPS & SUGGESTIONS

- Recruiters often read the README before the code - make it shine.
- Add a 'What I learned / would improve' section; it signals maturity.
- Keep behavioral stories concrete and outcome-focused.

DAY 60

Final Revision & Job Applications

WHAT YOU'LL LEARN

- **Final revision:** Review your weak areas and quiz answers one more time. Skim your own project code to refresh details.
- **Resume & LinkedIn:** Update your resume and LinkedIn with your projects, skills (Dart, Flutter, state management, REST, Firebase, testing) and links.
- **Publishing:** Optionally publish your app to the Play Store and/or App Store, or deploy the web build - a live link is powerful.
- **Applying:** Start applying, track your applications, and keep building small things to stay sharp.

DAILY PROJECT · Launch Day**WHAT YOU'LL BUILD**

Ship and apply: publish or deploy your app, finalize your resume and profiles, and begin sending applications.

STEPS TO DO

1. Polish and publish/deploy your capstone (store or web).
2. Update resume and LinkedIn with links and a short pitch.
3. Make a list of target roles/companies and apply to several.
4. Set up a simple tracker for applications and follow-ups.

TIPS & SUGGESTIONS

- A single live, polished app beats many half-finished demos.
- Tailor your resume to mention the exact skills in the job post.
- Keep a daily coding habit while job hunting - momentum matters.

Part 2 · Flutter & Dart Interview Questions

45 commonly asked questions with clear, concise answers.

Q1. What is Flutter and how does it work?

- Flutter is Google's open-source UI toolkit for building natively compiled apps for mobile, web, desktop and embedded from a single Dart codebase.
- It does not use native OEM widgets or a WebView. Instead it ships its own rendering engine and paints every pixel itself, giving consistent UI across platforms.
- Dart is compiled ahead-of-time (AOT) to native machine code for release builds, and uses just-in-time (JIT) compilation in debug to enable hot reload.
- The modern engine (Impeller) renders the widget tree to the screen via the GPU.

Q2. What is Dart and why does Flutter use it?

- Dart is a client-optimized, object-oriented, statically-typed language by Google.
- It supports both JIT (fast hot reload during development) and AOT (fast, predictable native code in production).
- It is single-threaded with an event loop and isolates, which suits UI programming well.
- Its sound null safety and rich async support (Future/Stream) make UI and data code safe and expressive.

Q3. Difference between StatelessWidget and StatefulWidget?

- A StatelessWidget is immutable: it has no internal mutable state and only depends on its configuration (constructor inputs).
- A StatefulWidget has a companion State object that can change over time; calling setState() rebuilds the widget.
- Use Stateless when the UI only depends on inputs; use Stateful when the widget must react to internal changes (input, animation, async data).

Q4. What does setState() do?

- setState() notifies the framework that the internal state has changed and the widget needs to be rebuilt.
- You mutate state variables inside the callback passed to setState; Flutter then schedules a rebuild of that widget's subtree.
- It only triggers a rebuild of the State's build method - it is local state, not global.

```
void _increment() {  
  setState(() {  
    _counter++;  
  });  
}
```

Q5. Explain the widget tree, element tree and render tree.

- Widget tree: immutable configuration / blueprint you write in build().
- Element tree: the mutable instantiation of widgets; elements hold state and the link between widgets and render objects, and manage rebuilds efficiently.
- Render tree (RenderObject): handles layout, painting and hit-testing.
- Keeping widgets cheap and immutable lets Flutter diff them and reuse elements/render objects efficiently.

Q6. What is BuildContext?

- BuildContext is a handle to the location of a widget in the element tree.
- It is used to look up inherited widgets (e.g., Theme.of(context), MediaQuery.of(context)) and to navigate (Navigator.of(context)).
- Each widget has its own context; the context you use matters when looking things up the tree.

Q7. What are Keys and when do you need them?

- Keys preserve widget state when the position of widgets in a list changes.
- Without keys, Flutter matches widgets by type and position; reordering stateful widgets can cause state to attach to the wrong item.
- Common keys: `ValueKey`, `ObjectKey`, `UniqueKey` and `GlobalKey` (which also gives access to state/context).

Q8. Difference between hot reload and hot restart?

- Hot reload injects updated source into the running Dart VM and rebuilds the widget tree, preserving app state - it is very fast.
- Hot restart restarts the app, resetting all state, but re-running `main()`.
- Hot reload doesn't run `main()/initState` again; structural changes (e.g., to enums, generics, static fields) may require a hot restart.

Q9. Explain sound null safety in Dart.

- By default types are non-nullable; a `String` can never be null unless declared `String?`.
- This moves a whole class of null-reference errors from runtime to compile time.
- Tools: `?` (nullable), `!` (assert non-null), `late` (initialized later), `??` and `?.` for null-aware access.

```
String? name;           // can be null
String city = 'Pokhara'; // cannot be null
int len = name?.length ?? 0;
```

Q10. Difference between final and const in Dart.

- `final` means the variable is assigned once and cannot be reassigned, but the value is determined at runtime.
- `const` means a compile-time constant; the value must be known at compile time and is canonicalized (shared).
- `const` objects (e.g., `const` widgets) help Flutter skip rebuilds and save memory.

```
final now = DateTime.now(); // runtime
const pi = 3.14159;         // compile-time
```

Q11. What are Future and Stream?

- A `Future` represents a single asynchronous value that completes once (success or error).
- A `Stream` represents a sequence of asynchronous events over time (zero or more values).
- Use `async/await` with `Futures`; use `await for` or `StreamBuilder` with `Streams`.

```
Future<int> fetch() async => 42;
Stream<int> ticks() async* {
  for (var i = 0; i < 3; i++) yield i;
}
```

Q12. Explain async, await and the event loop.

- `async` marks a function that returns a `Future`; `await` pauses execution until the awaited `Future` completes, without blocking the thread.
- Dart is single-threaded with an event loop; while awaiting, other events keep being processed.
- There are two queues: microtasks (higher priority) and events; `await` schedules continuation as a microtask/event.

Q13. What is an Isolate?

- Isolates are Dart's unit of concurrency; each has its own memory and event loop - no shared mutable state.
- They communicate by passing messages over ports, avoiding locks and data races.
- Use `compute()` or `Isolate.run()` to move heavy CPU work (e.g., JSON parsing of large payloads) off the UI isolate.

Q14. FutureBuilder vs StreamBuilder?

- `FutureBuilder` rebuilds based on the state of a single `Future` (one-shot async data).
- `StreamBuilder` rebuilds for each event emitted by a `Stream` (continuous data).
- Both expose an `AsyncSnapshot` with `connectionState`, `data` and `error` - handle all states.
- Tip: create the `Future/Stream` in `initState`, not inside `build`, to avoid re-fetching on every rebuild.

Q15. How does navigation work in Flutter?

- The `Navigator` manages a stack of routes (screens).
- Imperative: `Navigator.push/pop` with `MaterialPageRoute`; data goes in via constructors and returns via the popped value.
- Declarative / URL-based routing is handled well by packages like `go_router`, which suit deep links and web.

```
Navigator.push(context,
  MaterialPageRoute(builder: (_) => DetailScreen(id: 5)));
```

Q16. Compare popular state management solutions.

- `setState`: built-in, simplest, best for local widget state.
- `Provider`: lightweight `InheritedWidget` wrapper using `ChangeNotifier`; great default for many apps.
- `Riverpod`: compile-safe, testable, no `BuildContext` needed to read providers; a modern evolution of `Provider`.
- `Bloc/Cubit`: event-driven, predictable, strong for large teams and complex flows.
- `GetX/MobX`: also popular; choose based on team familiarity and app complexity.

Q17. What is Provider and ChangeNotifier?

- `ChangeNotifier` is a simple class that holds state and calls `notifyListeners()` when it changes.
- `ChangeNotifierProvider` exposes it to the widget tree; widgets read it via `context.watch/read` or `Consumer`.
- `context.watch` rebuilds on change; `context.read` reads once without listening (good for callbacks).

```
class Counter extends ChangeNotifier {
  int count = 0;
  void increment() { count++; notifyListeners(); }
}
```

Q18. What are the StatefulWidget lifecycle methods?

- `createState()` - creates the `State` object.
- `initState()` - one-time init (subscriptions, controllers).
- `didChangeDependencies()` - when an inherited dependency changes.
- `build()` - describes the UI; can run many times.
- `didUpdateWidget()` - when the parent rebuilds with a new config.
- `dispose()` - cleanup (controllers, streams) - always dispose to avoid leaks.

Q19. Difference between mainAxisAlignment and crossAxisAlignment?

- For a `Row`, the main axis is horizontal and the cross axis is vertical; for a `Column` it is the opposite.
- `mainAxisAlignment` positions children along the main axis (start, center, spaceBetween, etc.).
- `crossAxisAlignment` positions them along the cross axis (start, center, stretch, etc.).

Q20. Explain Flutter's constraint model.

- The rule of thumb: 'Constraints go down, sizes go up, parent sets position.'
- A parent passes constraints (min/max width and height) to a child; the child chooses its size within them; the parent then positions it.
- Widgets like Expanded, Flexible and SizedBox influence how constraints are distributed.

Q21. ListView vs ListView.builder?

- ListView (default) builds all children eagerly - fine for a small, fixed number of items.
- ListView.builder lazily builds items on demand as they scroll into view - essential for long or infinite lists for performance.
- Use itemCount and itemBuilder; reuse offscreen widgets efficiently.

Q22. How do you make API calls and parse JSON?

- Use the http or dio package to perform HTTP requests and get a response body.
- Decode with jsonDecode to get a Map/List, then map it into typed models via fromJson factory constructors.
- For larger projects, generate serialization with json_serializable or use freezed for immutable models.

```
final res = await http.get(Uri.parse(url));  
final data = jsonDecode(res.body);  
final user = User.fromJson(data);
```

Q23. What is the Repository pattern in Flutter apps?

- A repository abstracts data sources (REST, local DB, cache) behind a clean interface.
- The UI/state layer talks to the repository, not directly to http or the database.
- It improves testability (mock the repository) and makes swapping data sources easy.

Q24. How do you handle errors and loading states in the UI?

- Model UI as states: loading, data, empty and error.
- With FutureBuilder/StreamBuilder, check connectionState and snapshot.hasError.
- With state management, expose an enum/sealed state or AsyncValue and render accordingly.
- Show spinners for loading, friendly messages for errors, and a retry action.

Q25. Difference between named, positional and optional parameters in Dart.

- Positional: order matters - foo(1, 2).
- Named: passed by name in {} - foo(width: 1, height: 2); improves readability.
- Optional positional use [] and may have defaults; named can be marked required.

```
void box({required double w, double h = 10}) {}  
box(w: 5);
```

Q26. What is a mixin and how does it differ from inheritance?

- A mixin lets a class reuse code from multiple sources without classic single inheritance.
- You apply mixins with the with keyword; a mixin can restrict usage with an on clause.
- Mixins are for sharing behavior; inheritance models an is-a relationship.

```
mixin Logger { void log(String m) => print(m); }  
class Service with Logger {}
```

Q27. Abstract class vs interface in Dart.

- Dart has no separate interface keyword - every class implicitly defines an interface.
- An abstract class can have implemented methods and fields and is meant to be extended.
- When you use implements, you must provide implementations for everything (contract only).
- Dart 3 adds class modifiers: abstract, interface, base, final, sealed for stronger guarantees.

Q28. What is the difference between == and identical()?

- == compares values and can be overridden (along with hashCode) for value equality.
- identical() checks whether two references point to the exact same object.
- const objects with the same value are canonicalized, so identical() can be true for them.

Q29. How do you optimize Flutter performance?

- Use const constructors so unchanged subtrees are not rebuilt.
- Use ListView.builder for long lists; avoid heavy work in build().
- Split large widgets; use keys appropriately; cache images.
- Profile with DevTools (rebuild counts, raster/UI thread, jank) and avoid rebuilding the whole tree on small changes.

Q30. What are implicit vs explicit animations?

- Implicit animations (AnimatedContainer, AnimatedOpacity) animate automatically when their properties change - simple and declarative.
- Explicit animations use an AnimationController + Tween for full control over timing, direction and replay.
- Explicit animations need a TickerProvider (vsync) and must dispose the controller.

Q31. How do you persist data locally?

- shared_preferences: small key-value data (settings, flags, tokens).
- sqflite: a SQLite relational database for structured data.
- Hive / Isar: fast NoSQL/object stores, good for offline-first apps.
- Files via path_provider for documents/cache directories.

Q32. What is pubspec.yaml?

- It is the project's manifest: name, version, environment SDK constraints.
- It declares dependencies and dev_dependencies (packages from pub.dev).
- It registers assets (images, fonts) so they can be bundled and loaded.

Q33. Difference between dependencies and dev_dependencies?

- dependencies are needed at runtime and ship with the app (e.g., http, provider).
- dev_dependencies are only used during development (e.g., build_runner, flutter_test, lints).
- Keeping dev tools out of runtime keeps the app smaller and cleaner.

Q34. What are extension methods?

- Extensions add new functionality to existing types without subclassing or modifying them.
- Great for utility helpers on String, int, BuildContext, etc.
- They are resolved at compile time based on the static type.

```
extension Cap on String {
  String get capitalized =>
    isEmpty ? this : this[0].toUpperCase() + substring(1);
}
```

Q35. Explain the SOLID principles.

- Single Responsibility: a class should have one reason to change.
- Open/Closed: open for extension, closed for modification.
- Liskov Substitution: subtypes must be usable in place of their base type.
- Interface Segregation: prefer small, focused interfaces.
- Dependency Inversion: depend on abstractions, not concrete implementations.

Q36. What is dependency injection and how is it done in Flutter?

- DI means providing a class's dependencies from the outside rather than creating them internally.
- It improves testability and decoupling.
- In Flutter it's commonly done via Provider/Riverpod, get_it (service locator), or constructor injection.

Q37. How do you test a Flutter app?

- Unit tests verify pure Dart logic (functions, models, services).
- Widget tests verify UI in isolation using testWidgets, pumpWidget, finders and matchers.
- Integration tests run the full app on a device/emulator.
- Use mocktail/mockito to mock dependencies.

```
testWidgets('shows title', (tester) async {
  await tester.pumpWidget(const MyApp());
  expect(find.text('Home'), findsOneWidget);
});
```

Q38. What is the difference between WidgetsApp, MaterialApp and CupertinoApp?

- WidgetsApp is the low-level base providing routing, localization and core setup.
- MaterialApp builds on it adding Material Design theming, widgets and navigation.
- CupertinoApp provides iOS-style (Cupertino) design language and widgets.

Q39. What are the differences among AddSingleton-style lifetimes in get_it / DI?

- Singleton/lazySingleton: one shared instance for the app's lifetime.
- Factory: a new instance each time it is requested.
- Scoped instances live for the duration of a particular scope (e.g., a feature/session).
- Pick based on whether state should be shared or fresh.

Q40. How do you handle responsive and adaptive UI?

- Use MediaQuery for screen size/orientation and LayoutBuilder for parent constraints.
- Use flexible widgets (Expanded, Flexible, FractionallySizedBox) and breakpoints.
- Adaptive widgets switch between Material and Cupertino based on platform.
- Packages like flutter_screenutil or responsive_framework can help.

Q41. What is the difference between Navigator 1.0 and 2.0?

- Navigator 1.0 is imperative (push/pop) - simple and great for most apps.
- Navigator 2.0 is declarative: the route stack is derived from app state, enabling deep links and full web URL control.
- `go_router` wraps Navigator 2.0 with a much simpler API.

Q42. How do you reduce app size and improve startup?

- Build release with AOT; use `--split-debug-info` and obfuscation.
- Use app bundles (.aab) so stores deliver per-device assets.
- Compress images, remove unused assets/packages, and defer heavy work after first frame.

Q43. What is the difference between Container and SizedBox?

- `SizedBox` is a lightweight box used purely for sizing or spacing (const-friendly).
- `Container` is a convenience widget combining padding, margin, color, decoration, constraints and alignment.
- Prefer `SizedBox` for simple spacing for performance.

Q44. What are common causes of unnecessary rebuilds and how to avoid them?

- Putting state too high in the tree so large subtrees rebuild.
- Not using `const` for static widgets.
- Rebuilding with `Provider`'s `watch` where `read`/`Selector` would do.
- Solutions: localize state, use `const`, use `Selector`/`Consumer` narrowly, and split widgets.

Q45. How do you deploy a Flutter app?

- Android: build an app bundle (`flutter build appbundle`) and upload to Google Play.
- iOS: build via Xcode (`flutter build ipa`) and submit through App Store Connect.
- Web: `flutter build web` and host the output (Firebase Hosting, Netlify, etc.).
- Desktop: build platform-specific bundles for Windows/macOS/Linux. CI/CD (Codemagic, GitHub Actions) automates this.

Part 3 · Quizzes & Answer Keys

Checkpoints to test yourself at the end of each week.

Quiz 1 - Dart Fundamentals (Week 1)

1. Which keyword declares a compile-time constant in Dart?

- A) final
- B) const
- C) static
- D) late

2. What is the entry point of every Dart program?

- A) start()
- B) run()
- C) main()
- D) init()

3. Which operator returns the right value only when the left is null?

- A) ?.
- B) ??
- C) !
- D) ..=

4. Which collection stores only unique values?

- A) List
- B) Map
- C) Set
- D) Queue

5. What does the ~/ operator do?

- A) Modulo
- B) Integer division
- C) Power
- D) Bitwise NOT

6. A variable that may hold null must be declared as:

- A) String!
- B) String?
- C) String*
- D) String&

7. Which loop is guaranteed to run at least once?

- A) for
- B) while
- C) do-while
- D) for-in

Answer key: 1-B 2-C 3-B 4-C 5-B 6-B 7-C

Quiz 2 - Dart OOP & Advanced (Week 2)

1. Which keyword is used to inherit from a class?
 - A) implements
 - B) with
 - C) extends
 - D) on
2. A constructor that can return an existing/cached instance is a:
 - A) named constructor
 - B) factory constructor
 - C) const constructor
 - D) default constructor
3. To reuse code across multiple classes without inheritance you use a:
 - A) interface
 - B) mixin
 - C) enum
 - D) typedef
4. Which represents a single asynchronous value?
 - A) Stream
 - B) Future
 - C) Iterable
 - D) Isolate
5. Which keyword yields multiple async values from a generator?
 - A) return
 - B) await
 - C) yield
 - D) throw
6. Dart's unit of concurrency with isolated memory is the:
 - A) Thread
 - B) Task
 - C) Isolate
 - D) Process
7. To catch a specific exception type you use:
 - A) catch (e)
 - B) on Type catch (e)
 - C) finally
 - D) rethrow

Answer key: 1-C 2-B 3-B 4-B 5-C 6-C 7-B

Quiz 3 - Flutter Basics (Week 3)

1. Which widget should you use when the UI never changes internally?

- A) StatefulWidget
- B) StatelessWidget
- C) InheritedWidget
- D) RenderObject

2. Which method triggers a rebuild in a StatefulWidget?

- A) rebuild()
- B) setState()
- C) notify()
- D) update()

3. Which widget arranges children horizontally?

- A) Column
- B) Stack
- C) Row
- D) ListView

4. For a long, scrollable list you should use:

- A) ListView (default)
- B) Column
- C) ListView.builder
- D) Wrap

5. Which widget provides the basic Material screen structure?

- A) MaterialApp
- B) Scaffold
- C) Container
- D) SafeArea

6. Hot reload preserves which of the following?

- A) Nothing
- B) App state
- C) Only widgets
- D) Only routes

7. Where do you register assets like images and fonts?

- A) main.dart
- B) pubspec.yaml
- C) AndroidManifest.xml
- D) analysis_options.yaml

Answer key: 1-B 2-B 3-C 4-C 5-B 6-B 7-B

Quiz 4 - Navigation, Forms & State (Week 4)

1. Which class manages the stack of screens?

- A) Router
- B) Navigator
- C) RouteStack
- D) PageManager

2. To validate a Form you typically use a:

- A) GlobalKey
- B) GlobalKey<FormState>
- C) UniqueKey
- D) ObjectKey

3. Which reads a Provider value without listening for changes?

- A) context.watch
- B) context.read
- C) Consumer
- D) Selector

4. Lifting state up means:

- A) Moving state into a database
- B) Moving shared state to a common ancestor
- C) Using global variables
- D) Deleting state

5. A package for declarative, URL-based routing is:

- A) dio
- B) go_router
- C) hive
- D) intl

6. ChangeNotifier signals listeners by calling:

- A) setState()
- B) notifyListeners()
- C) rebuild()
- D) emit()

7. TextEditingController is mainly used to:

- A) Style text
- B) Read/control a text field's value
- C) Validate forms
- D) Navigate

Answer key: 1-B 2-B 3-B 4-B 5-B 6-B 7-B

Quiz 5 - State Management & Networking (Week 5)

1. Which package is commonly used for simple HTTP requests?

- A) http
- B) sqflite
- C) intl
- D) provider

2. Which converts a JSON string into a Dart Map?

- A) jsonEncode
- B) jsonDecode
- C) toJson
- D) fromJson

3. Which widget rebuilds for each event of a Stream?

- A) FutureBuilder
- B) StreamBuilder
- C) ListView
- D) Consumer

4. A common pitfall is creating a Future inside:

- A) initState
- B) build
- C) a repository
- D) a service

5. Which state solution needs noBuildContext to read providers?

- A) setState
- B) Provider
- C) Riverpod
- D) InheritedWidget

6. Bloc reacts to incoming ____ and emits ____.

- A) states / events
- B) events / states
- C) actions / reducers
- D) props / state

7. The pattern that hides data sources behind one interface is the:

- A) Singleton
- B) Repository
- C) Observer
- D) Factory

Answer key: 1-A 2-B 3-B 4-B 5-C 6-B 7-B

Quiz 6 - Advanced Flutter (Week 6)

1. For small key-value settings you use:

- A) sqflite
- B) shared_preferences
- C) dio
- D) freezed

2. Which is a fast NoSQL local database for Flutter?

- A) Hive
- B) Postgres
- C) Redis
- D) MongoDB

3. Animations that animate automatically on property change are:

- A) explicit
- B) implicit
- C) manual
- D) physics-based

4. Explicit animations require a controller plus a:

- A) Stream
- B) Tween
- C) Future
- D) Isolate

5. Which test type uses testWidgets and pumpWidget?

- A) unit test
- B) widget test
- C) integration test
- D) golden test

6. Using const widgets primarily helps with:

- A) security
- B) performance / fewer rebuilds
- C) networking
- D) theming

7. Which command builds an Android app bundle?

- A) flutter run
- B) flutter build apk
- C) flutter build appbundle
- D) flutter test

Answer key: 1-B 2-A 3-B 4-B 5-B 6-B 7-C

Quiz 7 - Final Mixed Review (Weeks 1-6)

1. Flutter renders its UI by:

- A) Using native OEM widgets
- B) Embedding a WebView
- C) Painting pixels with its own engine
- D) Generating HTML

2. Release Flutter apps are compiled using:

- A) JIT only
- B) AOT to native code
- C) Bytecode interpreter
- D) Transpiling to JS

3. BuildContext primarily represents:

- A) The app theme
- B) A widget's location in the element tree
- C) The render object
- D) The navigation stack

4. Keys are most important when:

- A) Styling text
- B) Reordering stateful list items
- C) Making HTTP calls
- D) Writing tests

5. Which lifecycle method should release controllers/streams?

- A) initState
- B) build
- C) dispose
- D) didUpdateWidget

6. 'Constraints go down, sizes go up' describes Flutter's:

- A) state model
- B) layout/constraint model
- C) navigation model
- D) build modes

7. Which Dart 3 modifier enables exhaustive switch over a closed set of subtypes?

- A) abstract
- B) sealed
- C) base
- D) mixin

8. The best choice for a very long scrolling list is:

- A) Column
- B) ListView.builder
- C) Wrap
- D) Stack

Answer key: 1-C 2-B 3-B 4-B 5-C 6-B 7-B 8-B

You Made It - Now Keep Building

Sixty days is enough to become genuinely capable in Flutter - but only if you ship code every day. By now you should have a GitHub profile full of small projects and at least one polished capstone app.

Next steps: publish your capstone to the Play Store and/or App Store or deploy the web build, write a strong README with screenshots and a demo, and add the project to your resume and LinkedIn.

Keep growing: read other people's Flutter source, follow the official docs and release notes, contribute to an open-source package, and rebuild an early project with cleaner architecture to see how far you've come.

Consistency beats intensity. One focused hour every day will take you further than the occasional marathon. Good luck!